

# TECHNICAL REFERENCE & USER MANUAL

## Microservices Architecture

---

## 1 System Overview & Core Architecture

This document serves as the official reference guide and operational manual for the rebuilt version of the `WebDevStd2531` distributed system. It replaces all prior design representations to accurately reflect the decoupled architecture and current microservice topologies.

Unlike older monolithic or gateway-reliant prototypes, the current production environment relies on a balanced backend architecture containing four distinct, independent deployable service units, a shared persistent database storage model, and an asynchronous event broker.

### 1.1 Architectural Strategy

The system layout uses the **Backends for Frontends (BFF)** and service-isolation patterns:

- **WebDevStd2531 Web App:** Functions as the single-point-of-entry client wrapper and route coordinator, decoupling the UI rendering layer from core transactional logic.
- **UserService:** Manages identity boundaries, including authentication, registration, permission schemes, and cryptographic JWT issuance.
- **CatalogService:** Controls the inventory catalog, options, categories, and item state management.
- **OrderService:** Manages the lifecycle of user cart structures, item adjustments, and payment flows.

While individual services retain exclusive ownership over their respective database tables in code, deployment complexity has been optimized for scope and simplicity by routing all schemas through a shared physical SQL Server container. Synchronous integration is facilitated via high-performance gRPC channels, while asynchronous transactions publish domain events directly to RabbitMQ to achieve eventual consistency.

Table 1: Quick Architecture &amp; Role Mapping

| Service Name   | Operational Role                                             | Main Transport                              | Main Data Target                                |
|----------------|--------------------------------------------------------------|---------------------------------------------|-------------------------------------------------|
| WebDevStd2531  | Front-end UI, Wrapper/BFF & Router Coordinator               | MVC Controller with gRPC Client integration | Identity Database tables via shared DB instance |
| UserService    | User registration, login, role assignments & security tokens | gRPC Server endpoints                       | Identity tables inside SQL Server               |
| CatalogService | Product display, categorizations, and stock tracking         | gRPC Server endpoints                       | Catalog tables inside SQL Server                |
| OrderService   | Cart states, user adjustments & order pipelines              | gRPC Server endpoints                       | Order tables inside SQL Server                  |
| db             | Central data container                                       | SQL Server Wire                             | Relational schemas for all services             |
| rabbitmq       | Async messaging coordinator                                  | AMQP 0-9-1                                  | Event exchanges and queues                      |

## 2 System Communication Matrix

Synchronous service-to-service communication is achieved via gRPC unary and streaming channels, supplemented by secure JSON Web Token (JWT) credentials attached to request metadata. The communication topologies, protocols, and distinct business purposes are outlined in Table 2.

## 3 Service Boundary Analysis & Structural Architecture

Each service operates inside an isolated logical boundary, containing specific dependency models, schemas, and operational code bases. Below is a structured layout of key files, entry points, and functions making up the execution path.

### 3.1 WebDevStd2531: The BFF Wrapper

The MVC application functions as the user-facing interface, keeping the browser logic simple and routing actions back to the microservices.

- **Entry Point & Setup (Program.cs):** Orchestrates the service container, registers gRPC channels for the target microservices, establishes local authentication cookie handlers, and registers global background tasks like RabbitMQ event listeners.
- **Route Controllers:**
  - **HomeController:** Executes `Index()` to fetch homepage items and structural menus from the Catalog service.

Table 2: System Communication Matrix

| Source Service | Target         | Interaction Type                    | Protocol / Tool       | Business Purpose                                                                     |
|----------------|----------------|-------------------------------------|-----------------------|--------------------------------------------------------------------------------------|
| Main MVC app   | UserService    | Synchronous request/re-sponse       | gRPC unary            | Register, login, role-based auth, JWT issuance                                       |
| Main MVC app   | CatalogService | Synchronous request/re-sponse       | gRPC unary            | Load homepage products, product details, category pages, admin category/product CRUD |
| Main MVC app   | CatalogService | Streaming                           | gRPC server streaming | Stream product list data if needed later for live browse/feed-style UI               |
| Main MVC app   | OrderService   | Synchronous request/re-sponse       | gRPC unary            | Cart actions, checkout, payment completion                                           |
| Main MVC app   | OrderService   | Streaming                           | gRPC server streaming | Stream cart items if needed later for UI refresh or larger cart views                |
| OrderService   | CatalogService | Synchronous request/re-sponse       | gRPC unary            | Get product data, validate stock, update stock during checkout                       |
| UserService    | Database       | Direct data access                  | EF Core / SQL Server  | Store and manage Identity users, roles, claims                                       |
| CatalogService | Database       | Direct data access                  | EF Core / SQL Server  | Store products, categories, grand categories, options                                |
| OrderService   | Database       | Direct data access                  | EF Core / SQL Server  | Store orders and order products/cart state                                           |
| OrderService   | CatalogService | Synchronous service-to-service auth | JWT - gRPC metadata   | Trust service calls for stock-adjusting operations                                   |
| Main MVC app   | OrderService   | Synchronous service-to-service auth | JWT - gRPC metadata   | Trust cart/checkout calls from the wrapper app                                       |
| Main MVC app   | CatalogService | Synchronous service-to-service auth | JWT - gRPC metadata   | Trust admin write calls from the wrapper app                                         |

- **UserController:** Coordinates user onboarding via `Register()` and signs users in via `Login()`, exchanging plain credentials for secure token payloads.
  - **CartController:** Provides cart tracking (`CartDetail()`), updates stock quantities (`AddCartItem()`, `RemoveCartItem()`, `IncrCartItem()`, `DecrCartItem()`), and finishes orders (`Pay()`).
  - **ProdController & CateController:** Handle catalog details and search categories.
  - **Areas/Admin/Controllers/ProdAdminController & CateAdminController:** Safeguard CRUD write paths, restricting modifications to users with verified `Admin` roles.
- **gRPC Clients:** Managed classes (`UserAccountGrpcClient`, `CatalogGrpcClient`, and `OrderGrpcClient`) wrap raw connection logic, translate internal gRPC models to standard front-end view models, and handle exceptions.
  - **System Components:**
    - **CategoryNavigationViewComponent:** Loads global menus asynchronously via the `Catalog` client on layout renders.
    - **RabbitMqEventConsumerHostedService:** A long-running background worker monitoring the broker for backend event logs.

### 3.2 UserService: Authentication & Identity

The `UserService` acts as the primary authority for account management and security.

- **Entry Point (`Program.cs`):** Configures gRPC server pipelines, registers DB contexts, and binds encryption and publishing mechanisms.
- **Core Services & Data Objects:**
  - **UserAccountService:** Validates password hashes, processes registrations, maps user roles, and generates signed JWTs.
  - **AppUser & AppDbContext:** Represent security records mapped directly to the shared SQL database.
  - **JwtOptions:** Holds settings for asymmetric keys, expiration limits, and target audiences.
- **Contracts & Async Infrastructure:**
  - **Protos/user.proto:** Formally defines the RPC contracts (`RegisterUser`, `LoginUser`, and the streaming endpoint `StreamUsers`).
  - **RabbitMqEventPublisher:** Publishes the `UserRegistered` message asynchronously when new accounts are saved.

### 3.3 CatalogService: Inventory & Categories

The `CatalogService` manages the system catalog, handling details, listings, and updates.

- **Core Services & Data Objects:**
  - **CatalogGrpcService:** Handles read-write endpoints including homepage aggregation (`GetHomeCatalog`), category menus (`GetCategoryDetail`), inventory edits, and transactional updates like inventory deductions.
  - **CatalogDbContext, Product, Category, & GrandCategory:** Form the persistent product entities, keeping catalog logic decoupled from other domains.

- **Contracts & Async Infrastructure:**

- `Protos/catalog.proto`: Declares gRPC contracts including unary lookups, CRUD operations (`UpsertProduct`, `DeleteProduct`), and real-time data streaming (`StreamProducts`, `StreamCategories`).
- `RabbitMqEventPublisher`: Dispatches a `CatalogChanged` event whenever products, categories, or options are modified.

### 3.4 OrderService: Transactions & Checkout

The `OrderService` coordinates carts, items, payments, and checkout flows.

- **Core Services & Data Objects:**

- `OrderGrpcService`: Processes changes to item quantities (`AddCartItem`, `IncrementCartItem`, `DecrementCartItem`, and `RemoveCartItem`) and processes cart objects.
- `CatalogGrpcClient`: An internal client that queries the catalog for current prices and stock availability during checkout.
- `OrderDbContext`, `Order`, & `OrderProduct`: Track pending carts and finalized orders.

- **Contracts & Async Infrastructure:**

- `Protos/order.proto`: Defines cart RPCs, the `Pay` endpoint, and `StreamCartItems` for transactional data feeds.

## 4 Cross-Cutting Technical Concerns

### 4.1 gRPC Interface Definitions

The physical interaction contracts between services are defined using Protocol Buffers (Proto3).

Listing 1: Service Contract Outline

```
// UserService Interfaces
service UserAccount {
    rpc RegisterUser (RegisterRequest) returns (UserTokenResponse);
    rpc LoginUser (LoginRequest) returns (UserTokenResponse);
    rpc StreamUsers (Empty) returns (stream UserResponse);
}

// CatalogService Interfaces
service Catalog {
    rpc GetHomeCatalog (Empty) returns (HomeCatalogResponse);
    rpc GetProductDetail (ProductRequest) returns (ProductResponse);
    rpc GetCategoryDetail (CategoryRequest) returns (CategoryResponse);
    rpc UpsertProduct (ProductSaveRequest) returns (MutateResponse);
    rpc DeleteProduct (ProductDeleteRequest) returns (MutateResponse);
    rpc AdjustStock (StockAdjustmentRequest) returns (MutateResponse);
    rpc StreamProducts (Empty) returns (stream ProductResponse);
}

// OrderService Interfaces
service Order {
    rpc GetCart (CartRequest) returns (CartResponse);
    rpc AddCartItem (CartChangeRequest) returns (CartResponse);
    rpc RemoveCartItem (CartChangeRequest) returns (CartResponse);
    rpc IncrementCartItem (CartChangeRequest) returns (CartResponse);
    rpc DecrementCartItem (CartChangeRequest) returns (CartResponse);
    rpc Pay (PaymentRequest) returns (PaymentResponse);
    rpc StreamCartItems (Empty) returns (stream CartItemResponse);
}
```

### 4.2 JWT Security & Service Trust Propagation

To avoid checking the database on every microservice request, the system uses stateless JWT validation:

1. During authentication, UserService issues a signed JWT containing roles like Admin or User.
2. The client application (WebDevStd2531) captures this token and stores it in a secure HTTP-Only cookie.
3. For service-to-service calls (e.g., when the Web App calls OrderService or CatalogService to perform restricted writes), the JWT is attached to the gRPC request headers.
4. The receiving service validates the token offline using a shared cryptographic key, authorizing operations securely without extra database round-trips.

### 4.3 Logging & Monitoring Strategy

The system uses built-in ASP.NET core logging and health check libraries for tracking and diagnostics:

- **Logging:** Crucial events (such as logins, stock adjustments, cart changes, and RabbitMQ events) are captured via `ILogger` interfaces and printed to standard output. This output is aggregated by Docker's logging daemon.
- **Health Endpoints:** Standard `/health` endpoints check whether each service is running and can connect to its database.
- **Extended Monitoring:** The Web App exposes a `/db-health` page, offering an easy way to verify database reachability from a web browser.

## 4.4 RabbitMQ Messaging Infrastructure

The asynchronous messaging pattern uses a loose-coupling design:

- **Exchange:** Global events publish to the `webdevstd2531.events` exchange.
- **Queue Bindings:** The Web App consumes messages via the `webdevstd2531.web` queue.
- **Event Payload Types:**
  - `UserRegistered`: Dispatched on new account creations.
  - `CatalogChanged`: Dispatched on inventory updates or product modifications.
- **Error Resilience:** Message publishing is designed as best-effort. If RabbitMQ is offline, transactional workflows like checkout and registrations continue to execute, and errors are handled gracefully.

## 5 System Deployment & Topology

### 5.1 Docker Infrastructure

Every service runs in a container built from its own `Dockerfile`. This configuration allows services to be built and run independently, with orchestration managed via a `root/docker-compose.yml` file.

#### Port Configuration & Network Architecture

The system routes traffic using Docker's internal DNS naming. Internal services are isolated, and only necessary ports are exposed to the host machine:

- **Web UI Gateway:** External: 8080 → Internal Web Container: 8080
- **UserService:** External: 8081 → Internal Container: 8080
- **CatalogService:** External: 8082 → Internal Container: 8080
- **OrderService:** External: 8083 → Internal Container: 8080
- **SQL Server (db):** Port 1433
- **RabbitMQ AMQP Broker:** Port 5672 (Management Dashboard: 15672)

## 5.2 Startup Commands

To clean, build, and run the entire microservices stack, run the following commands in the root directory:

```
# Build image sets and launch the container network
docker compose up --build

# Run in background mode
docker compose up -d --build

# Verify container statuses and health
docker compose ps

# Follow container logs (example: Web UI)
docker compose logs -f web
```



## 6 Interactive User Manual & Verification Guide

This step-by-step verification guide walks through the system features and verifies that the microservices are integrated correctly.

### 6.1 User Workflows

1. **Storefront Browsing:** Open <http://localhost:8080>. The homepage should load products and dynamic menus. The Web App retrieves this data from `CatalogService` using unary gRPC calls.
2. **Account Registration & Logins:** Click **Sign Up** and register a new user. The Web App delegates the credentials to `UserService`. Once logged in, open the browser's developer tools and verify that the secure cookie is set. Then, log in with an admin account and check that administrative links appear in the navigation bar.
3. **Admin CRUD Operations:** Navigate to `Admin/ProdAdmin` or `Admin/CateAdmin`. Create a test product or category. Verify that modifications update the database, and verify that the Web App's console logs show the action.
4. **Cart & Order Flow:** Add products to your cart, navigate to `/Cart`, and adjust quantities. The Web App forwards these operations to `OrderService`, which queries `CatalogService` to check inventory levels. Finally, click **Checkout** to place the order, deduct inventory, and complete the transaction.

### 6.2 System Verification Steps

- **Observability Audits:** Open a terminal window and stream logs while using the application:

```
docker compose logs -f web userservice catalogservice orderservice
```

Verify that actions like user creation, item selection, and checkout log corresponding info messages.

- **Health Endpoints Testing:** Open the following URLs in a browser to check service health:
  - Web App Health: <http://localhost:8080/health>
  - Web DB Interface: <http://localhost:8080/db-health>
  - UserService Status: <http://localhost:8081/health>
  - CatalogService Status: <http://localhost:8082/health>
  - OrderService Status: <http://localhost:8083/health>
- **Fault-Tolerance Demonstration:** To test the system's resilience, stop the database container and check the health endpoints:

```
# Stop the database container
docker compose stop db

# Verify the Web App health endpoint returns a 503 error
curl -i http://localhost:8080/db-health

# Restart the database container and verify the service recovers
docker compose start db
```

- **Asynchronous Messaging Audit:** Open the RabbitMQ management dashboard at <http://localhost:15672> (Username: `guest` / Password: `guest`). Register a new user or modify catalog items, and watch the event message traffic on the chart. Then, check the Web App's console logs to confirm the listener received the background messages.

## 7 Technical FAQ & Common Troubleshooting

### Is an API Gateway part of this solution?

No. To simplify the architecture, this version intentionally omits a separate API Gateway. Instead, the `WebDevStd2531` MVC application acts as a direct client wrapper (BFF), communicating directly with the backend microservices over internal gRPC networks.

### Does every service run a separate database engine?

No. For development simplicity, the system runs a single SQL Server database container (db). However, service boundaries are strictly preserved in code: each service operates on its own dedicated database context and schemas. This design maintains logical isolation while simplifying container orchestration.

### How can I inspect the database structure directly?

To connect to the database from a local IDE or database tool (such as SQL Server Management Studio or Azure Data Studio), point your client to `localhost,1433` using the credentials configured in your root environment settings.

### Where should a developer start within the codebase?

To trace the application flows, start with these key entry points:

- **Routing & UI Entry Points:**

- `WebDevStd2531/Program.cs` (Dependency registrations)
- `WebDevStd2531/Controllers/HomeController.cs` (Storefront routing)
- `WebDevStd2531/Controllers/UserController.cs` (Identity routing)
- `WebDevStd2531/Controllers/CartController.cs` (Checkout routing)

- **Microservices Core Implementations:**

- `UserService/Services/UserAccountService.cs` (Identity management)
- `CatalogService/Services/CatalogGrpcService.cs` (Catalog management)
- `OrderService/Services/OrderGrpcService.cs` (Order processing)